Kalasalingam Academy of Research & Education

CAPTIONCRAFT AI

AI Powered Social Media Caption Studio

Project ID	PRJ030
Team Name	Tricky Techs
Team GitHub Repository	github.com/rukesh-r/social-media-caption-assistant

Registration No	Student Name	Individual GitHub Link
99230040904	TELLAPATI VENKATA KARTHIK	https://github.com/tellapatikarthik/ICL_GEN-AI_MASTERY
99230040883	RASETTY RUKESH	https://github.com/rukesh-r/Gen_AI_Mastery
99230040868	GONUGUNTLA RAMA KRISHNA	https://github.com/ramakrishnagonuguntla66-oss/ici-gen-ai
99230040877	MASIPEDDI NITHIN RAO	https://github.com/nithinraomasipeddi222/GEN-AI
99230041247	PAPPIREDDY DINESH KUMAR REDDY	https://github.com/pappireddydineshkumarreddy/Gen_AI_MASTERY.git

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

INTRODUCTION

1.1 Introduction

Social media platforms have become important communication and marketing channels for individuals, businesses, organizations and content creators. Creating effective captions requires understanding the target audience, selecting an appropriate tone, maintaining platform-specific formatting and producing engaging content.

Traditional caption creation depends heavily on manual writing. Users may also need different versions of the same message for different platforms. CaptionCraft AI addresses these challenges through an intelligent content generation platform powered by Generative AI.

The application uses a Large Language Model (LLM) to understand user instructions and generate meaningful captions. It also provides rewriting, quality analysis and document-based personalization. Unlike a basic caption generator, CaptionCraft AI combines multiple AI capabilities with a personal knowledge retrieval system.

1.2 Project Overview

CaptionCraft AI is a web-based application that provides four primary capabilities:

- Generate: Create captions using text topics or uploaded images.
- Rewrite: Transform existing captions into different writing styles.
- Analyze: Evaluate caption quality and suggest improvements.
- Knowledge RAG:Generate personalized captions using information from uploaded documents.

The application also provides user authentication and caption history management.

The project aims to make AI-powered content creation accessible, efficient and personalized. It focuses on reducing manual writing effort, generating content suitable for different platforms, improving caption quality, maintaining consistent brand communication and using specific business or product information during content generation.

1.3 Scope of the Project

- AI-powered text caption generation and image-based caption generation.
- Platform-specific content generation with tone, audience and length preferences.
- AI-powered caption rewriting and quality evaluation.
- Document-based knowledge retrieval and personalized generation.
- User authentication and caption history management.

1.4 Applications

- Social media content creators and digital marketing professionals.
- Small businesses, startups, brand managers and entrepreneurs.
- Educational institutions and organizations managing social media communication.

PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Creating engaging and relevant social media captions manually requires time, creativity and knowledge of the target audience. Basic caption generation approaches may produce generic content that does not always match a particular brand's identity, product information or communication style.

Users may also need separate tools to generate captions, improve existing content and evaluate writing quality. There is a need for an integrated intelligent system that can generate, rewrite, analyze and personalize social media captions while considering user preferences and relevant contextual information.

Therefore, CaptionCraft AI aims to develop a unified Generative AI-based platform that automates social media caption creation and improves content relevance through document-based knowledge retrieval.

2.2 Project Objectives

- Develop an intelligent social media caption-generation platform.
- Integrate a Large Language Model for natural language content generation.
- Generate captions for multiple social media platforms and support image based inputs.
- Provide different writing tones, audience selections and caption lengths.
- Develop AI-powered caption rewriting and quality evaluation.
- Implement a RAG pipeline for personalized content generation.
- Store user information, caption history and knowledge documents securely.
- Provide an accessible and user friendly interface.

2.3 Existing System

Traditional caption-generation methods generally involve manual content writing, searching for suitable hashtags, repeated editing, separate tools for content analysis and manually maintaining brand guidelines or product information.

2.3.1 Limitations of the Existing System

- Time-consuming content creation and repetitive writing efforts.
- Limited personalization and difficulty maintaining a consistent brand tone.
- Lack of integrated caption quality analysis.
- Limited use of private or user-provided document information.

2.4 Proposed System

CaptionCraft AI introduces an integrated Generative AI solution. Users can provide a topic, upload an image or submit an existing caption. The system processes the input and uses Gemini Flash Lite to generate or analyze content.

For personalized content generation, users can upload PDF, DOCX or TXT documents. The system extracts the information, creates embeddings and retrieves relevant document sections using vector similarity search. The retrieved information is supplied to the LLM as additional context to generate more relevant captions.

2.4.1 Advantages of the Proposed System

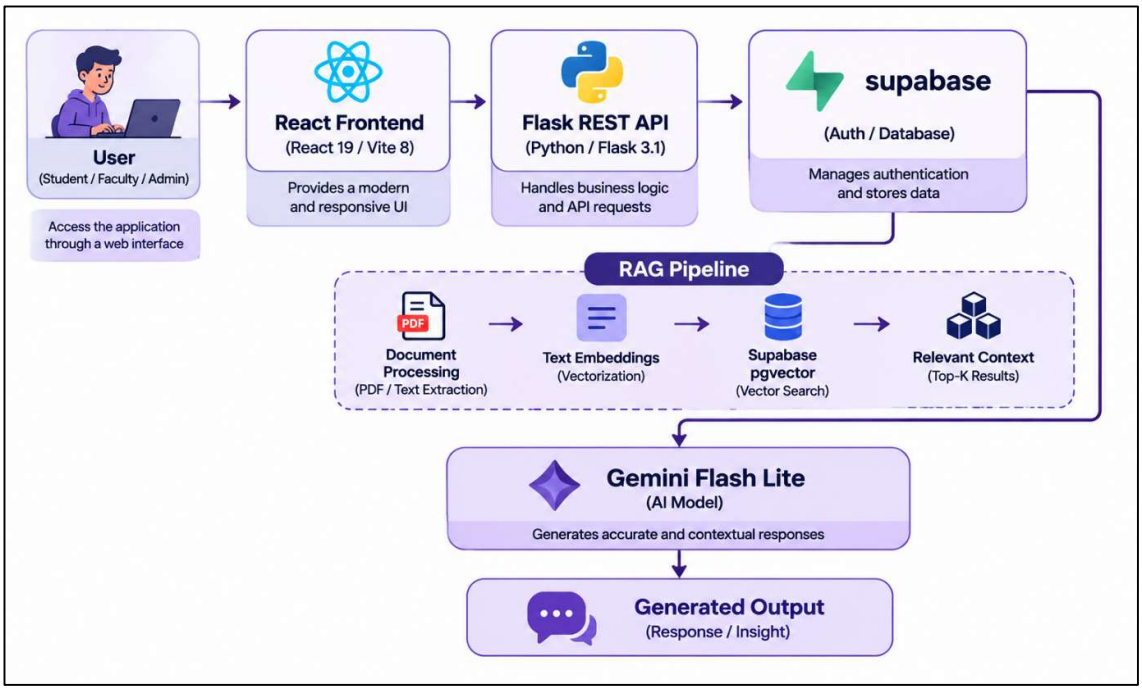
- Four platform caption generation.
- Multiple content tones and audience preferences.
- Image based caption generation.
- AI-powered rewriting and analysis using eight assessment metrics.
- Personalized generation using RAG.
- Centralized authentication and caption history.

SYSTEM ARCHITECTURE

3.1 Architecture Overview

CaptionCraft AI follows a full-stack architecture consisting of a frontend interface, backend REST API, Generative AI services and a database with vector search capabilities. The architecture separates user interaction, application processing, AI generation and data management.

3.1.1 High Level System Architecture



3.2 Frontend Architecture

The frontend is developed using React 19 and Vite 8. It provides an interactive interface for registration and login, text and image input, platform selection, tone and audience selection, caption rewriting, caption analysis, document management and caption history. Tailwind CSS is used for styling, while Axios handles communication with backend APIs.

3.3 Backend Architecture

The backend is developed using Flask 3.1. It handles authentication verification, API request processing, Gemini model integration, image and text input processing, caption generation and rewriting, caption quality analysis, document processing, embedding generation, vector retrieval and database operations.

3.4 Generative AI Architecture

The system integrates Google Gemini Flash Lite through the Google GenAI SDK. The model is used to generate captions from text topics and images, rewrite existing captions, analyze caption quality and generate personalized captions using retrieved document context.

Structured prompts guide the model according to the selected platform, tone, target audience and caption length.

3.5 Retrieval Augmented Generation (RAG) Architecture

Retrieval-Augmented Generation combines information retrieval with Large Language Model generation. It allows the application to use information from user-uploaded documents without training or fine-tuning its own LLM.

Stage 1 : Document Upload

The user uploads a PDF, DOCX or TXT document containing information about a brand, product, service or organization.

Stage 2 : Text Extraction

Text is extracted using PyMuPDF for PDF files, python-docx for DOCX files and text processing for TXT files.

Stage 3 : Text Chunking

The extracted content is divided into smaller text segments to make it easier to process and retrieve.

Stage 4 : Embedding Generation

Sentence Transformers converts text chunks into numerical vector embeddings. The database schema specifies a 384-dimensional embedding representation.

Stage 5 : Vector Storage

Text chunks, embeddings and metadata are stored in the Supabase knowledge chunks table.

Stage 6 : Query Processing

The user's request is converted into an embedding using the embedding model.

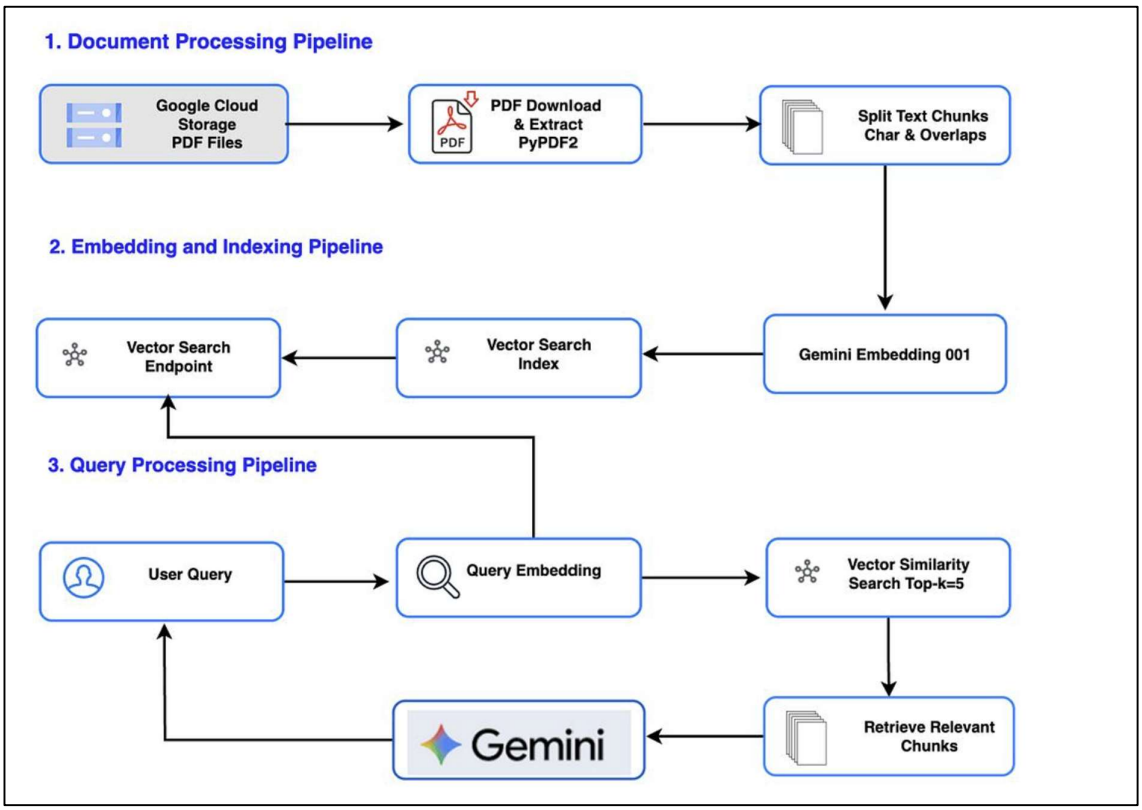
Stage 7 : Similarity Search

The system compares the query embedding with stored document embeddings and retrieves relevant chunks. The documented SQL function uses vector distance operations through pgvector.

Stage 8 : Context Aware Generation

Retrieved information is passed to Gemini Flash Lite as additional context. The model uses it with the user's instructions to generate a personalized caption.

3.5.1 RAG Workflow Diagram



3.5.2 Why RAG Is Used

A general-purpose LLM may not automatically know private or updated information about a specific brand or product. RAG retrieves relevant information from user-provided documents and uses it during generation. For example, if a user uploads a company's product document, the system can retrieve product specifications and use them when generating promotional captions.

RAG provides contextual grounding, but it does not guarantee that every generated statement is factually correct.

3.6 Database Architecture

Supabase is used for authentication and database management. The documented implementation contains two primary tables.

3.6.1 Caption History Table

This table stores user caption activity.

Field	Purpose
id	Unique record identifier
user_id	User reference
type	Operation type
platform	Social media platform
input_text	Original user input
result	Generated result
created_at	Creation timestamp

3.6.2 Knowledge Chunks Table

This table stores document chunks and their vector embeddings.

Field	Purpose
id	Unique record identifier
user_id	User reference
document_name	Uploaded document name
chunk_text	Extracted text segment
embedding	384-dimensional vector
metadata	Additional document information
created_at	Creation timestamp

The documented vector retrieval function applies user-specific filtering.

3.7 Security and Authentication

Supabase Auth is used for email and password authentication. The backend API uses Bearer token authentication for protected endpoints. This supports authenticated access to application resources and user-specific records.

IMPLEMENTATION DETAILS & TECH STACK

4.1 Technology Stack

4.1.1 Frontend Technologies

Technology	Purpose
React 19	User interface development
Vite 8	Development server and build tool
Tailwind CSS	Responsive UI styling
Axios	API communication
Supabase JavaScript SDK	Authentication and session handling

4.1.2 Backend Technologies

Technology	Purpose
Python	Backend programming
Flask 3.1	REST API development
Flask-CORS	Cross-origin request handling
Google GenAI SDK	Gemini model integration
Supabase Python SDK	Database operations
Gunicorn	WSGI application server

4.1.3 AI and Data Processing Technologies

Technology	Purpose
Gemini Flash Lite	Generative AI model
Retrieval-Augmented Generation	Context-aware content generation
Sentence Transformers	Text embedding generation
Supabase pgvector	Vector storage and similarity search
spaCy	NLP-based text processing
PyMuPDF	PDF text extraction
python-docx	DOCX text extraction

4.2 Module 1 – Caption Generation

This module generates captions based on user input. Input parameters include a topic or uploaded image, social media platform, writing tone, target audience and caption length. Supported platforms: Instagram, LinkedIn, Facebook and X.

The documented tone choices include Casual, Professional, Funny, Inspirational, Exciting and Promotional.

The generated output includes a main caption, three alternative hooks, ten suggested hashtags, emojis and a call to action.

4.3 Module 2 – Caption Rewriter

The Caption Rewriter improves existing social media content. Users submit a caption and choose a rewriting objective: More Engaging, Professional, Viral Humor, Inspirational, Punchy & Short or Emoji Polish. The system returns a refined caption, variations and suggested hashtags.

4.4 Module 3 – Caption Quality Analyzer

The analyzer evaluates an existing caption using AI. It generates a score between 0 and 100, assigns a quality grade and provides three actionable suggestions.

Metric	Description
Readability	Ease of understanding
Tone	Suitability of writing style
Engagement	Potential to engage readers
Hook Strength	Effectiveness of the opening
CTA	Quality of the call-to-action
Hashtag Quality	Relevance of hashtags
Clarity	Clear communication of the message
Audience Appeal	Suitability for the intended audience

4.5 Module 4 – Knowledge RAG

This module enables personalized caption generation using PDF, DOCX and TXT documents. The system extracts document content, divides it into chunks, generates embeddings and stores the information in the knowledge database. When a user requests a personalized caption, relevant information is retrieved and provided to Gemini as contextual input.

4.6 Module 5 – Caption History

This module stores the user's generated, rewritten and analyzed captions. Users can view previous activities, access saved results, review content history and delete individual history entries. Records are associated with the authenticated user in Supabase.

4.7 API Implementation

The Flask backend exposes REST API endpoints to connect frontend actions with backend services.

Method	Endpoint	Function
GET	/	Backend health check
POST	/generate-caption	Text-based caption generation
POST	/generate-image-caption	Image-based caption generation
POST	/rewrite-caption	Caption rewriting
POST	/analyze-caption	Caption quality analysis
GET	/history	Retrieve caption history
DELETE	/history/id	Delete a history record
POST	/rag/upload	Upload and process documents
GET	/rag/documents	List uploaded documents
DELETE	/rag/delete	Delete a document
POST	/rag/retrieve	Retrieve relevant document chunks
POST	/rag/generate	Generate personalized captions

Protected endpoints require an Authorization Bearer token.

4.8 Overall Application Workflow

1. The user registers or logs into the application.
2. The user selects a required functionality.
3. The user provides text, an image or an uploaded document.
4. The frontend sends the request to the Flask backend.
5. The backend processes the input and invokes the relevant AI or retrieval service.
6. For RAG requests, relevant document chunks are retrieved and supplied as context to Gemini.
7. The generated response is returned to the frontend.
8. Relevant caption activity is stored in Supabase.
9. The user views the result and can access previous caption history.

RESULTS AND TESTING

5.1 Project Outcomes

Caption Craft AI brings multiple Generative AI capabilities into one integrated application. The following indicators describe the documented feature scope and are not performance benchmarks.

Project Indicator	Implemented Scope
Social media platforms	4
Caption generation input types	Text and image
Supported writing tones	6
Caption rewriting objectives	6
Caption analysis metrics	8
Alternative caption hooks	3 per generation
Suggested hashtags	10 per generation
Improvement suggestions	3 per analysis
Document formats for RAG	3
Main database tables	2

5.2 Functional Testing

The following test cases provide a structured validation plan. The Pass/Fail result should be completed after executing each test in the deployed or local application.

Test ID	Test Scenario	Expected Result	Actual Status
TC01	User registration	User account is created	Working
TC02	User login	Authenticated access is provided	Working
TC03	Generate caption from text	Caption is generated	Working
TC04	Generate caption from image	Image-based caption is generated	Working
TC05	Select different platforms	Platform-specific content is returned	Working
TC06	Rewrite an existing caption	Rewritten caption and variations are returned	Working
TC07	Analyze caption	Score, grade and metrics are displayed	Working
TC08	Upload PDF document	Text is extracted and processed	Working
TC09	Upload DOCX/TXT document	Document content is processed	Working
TC10	Generate RAG caption	Relevant context is used during generation	Working
TC11	View caption history	Previous user activity is displayed	Working
TC12	Delete history entry	Selected history entry is removed	Working

5.3 Expected System Behaviour

- Generate captions based on user preferences.
- Adapt content for different social media platforms.
- Improve existing captions through AI rewriting.
- Identify caption quality issues.
- Retrieve relevant information from uploaded documents.
- Produce context-aware personalized captions.
- Maintain individual user caption history.

5.4 Sample Use Case

Scenario: A startup wants to promote a new product on Instagram.

10. The user uploads a product information PDF.
11. The application extracts product details and stores processed document information.
12. The user selects Instagram, Promotional tone and Customers as the target audience.
13. The user requests a personalized caption.
14. The RAG module retrieves relevant product information.
15. Gemini receives the user prompt and retrieved context.
16. The application generates a promotional caption with hooks, hashtags and a call-to-action.

Outcome: The user receives content based on product information rather than relying only on a generic prompt.

5.5 Project Significance

The project integrates text and image-based generation, content transformation, AI-based quality evaluation, document-based knowledge retrieval, vector embeddings, semantic search and user-specific content management. This demonstrates a practical application of Generative AI, LLM integration and RAG in a content creation scenario.

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

CaptionCraft AI demonstrates how Generative AI can be applied to social media content creation. The application combines React, Flask, Gemini Flash Lite and Supabase to provide an integrated platform for generating, rewriting and analyzing social media captions.

The implementation of Retrieval-Augmented Generation adds personalization by allowing users to upload their own documents and generate content using retrieved information. The project demonstrates practical use of Large Language Models, prompt engineering, text embeddings, vector similarity search, document processing and full-stack application development.

Overall, CaptionCraft AI provides a unified approach to intelligent social media content creation and demonstrates the potential of combining Generative AI with retrieval-based knowledge systems.

6.2 Final Project Outcome

CaptionCraft AI demonstrates a practical implementation of Generative AI through a full-stack application that combines content generation, content analysis and personalized knowledge retrieval.

The project moves beyond simple AI text generation by integrating an LLM with a document-based RAG pipeline, providing users with more context-aware and personalized social media content.